Runtime Membership Contracts for SWI-PeTTa

Research draft

6 November 2026

Abstract

PeTTa compiles MeTTa to SWI-Prolog. In the runtime typing draft described here, a declaration such as `(: f (-> A B C))` is a membership contract: a call of `f` is admitted only when its arguments belong to `A` and `B`, and its result must belong to `C`. Checking never runs an equation body a second time and never binds a program variable. An argument that is still unbound becomes a constraint that is checked when it binds. This note gives the judgments, the reading of `Atom` and of type parameters, the generated code, what the Lean model proves, and measurements against the current stable branch: every typed workload measured costs fewer inferences and less CPU time. It is a research draft, not a verified compiler or a static type system.

1 Contracts on values

PeTTa programs manipulate symbolic terms, unify variables, enumerate alternative answers and edit spaces while they run. A type check at a function boundary must therefore accept partially known data, preserve sharing, and fail rather than add or repeat answers. Reading each declaration as a predicate on values meets these requirements:

```
(: Nil (List $a))
(: Cons (-> $a (List $a) (List $a)))
(: same (-> $a $a Bool))
(= (same $x $y) True)
```

A call `(same 1 2)` is admitted because both arguments are numbers, and `(same 1 "s")` is refused because no single type contains both. The repeated `$a` asks for one common type; it says nothing about how `same` is implemented. Constructor parameters are fresh at each constructor node, whereas the parameters of a function signature have one assignment per call.

The design follows polymorphic logic programming [1] and the separation of goals from constraints in constraint logic programming [2]. Declarations guard execution at run time; they do not prove equation bodies well typed in advance. SWI-Prolog supplies the mechanisms: indexed clauses for membership tests, unification for private type equations, attributed variables for requirements that wait for a value [3], and `freeze/2` for independent primitive tests [4]. Enforcement reads the declarations directly. Reflection through `get-type` stays separate, so applications may extend it without changing what a contract admits.

2 Membership and type parameters

Values v are finite first-order terms; open values may also contain program variables. Γ is a declaration view. A type p is built from `Number`, `String`, `Bool`, `Symbol`, the structural forms `ListOf`, `TupleOf` and `PairOf`, declared families such as `(List $a)`, function types, and `Atom`, which inside a type is the type of every value. A well-formed type is *proper* if it does not mention `Atom`. Write $\Gamma \vdash v \Leftarrow p$ when the closed value v belongs to the closed type p. Representative rules are:

$$\frac{v \text{ is represented as a number}}{\Gamma \vdash v \Leftarrow \mathsf{Number}} \qquad \frac{}{\Gamma \vdash v \Leftarrow \mathsf{Atom}} \qquad \frac{\Gamma \vdash v_i \Leftarrow p \quad (1 \le i \le n)}{\Gamma \vdash [v_1, \ldots, v_n] \Leftarrow \mathsf{ListOf}(p)}$$

$$\frac{c : p_1, \ldots, p_n \to K(\bar{q}) \in \Gamma \quad \Gamma \vdash v_i \Leftarrow \rho p_i \ (1 \le i \le n)}{\Gamma \vdash c(v_1, \ldots, v_n) \Leftarrow K(\rho \bar{q})}.$$

A nominal fact `(: c T)` gives membership of the whole value `c`; it says nothing about fields. In the constructor rule, ρ instantiates the constructor's own parameters jointly, so repeated parameters stay equal. Formation is a premise: `(ListOf ListOf)` is malformed and admits no value, not even `()`. Primitive types

test the representation; a declaration cannot make a symbol a `Number`. An atom with no proper declaration is a `Symbol`. Recursive families whose type grows at each step are checked by the general relation instead of generated clauses.

Where Atom appears. `Atom` has different roles in a signature and inside a type:

Position	Meaning
Whole input slot, `(-> Atom ...)`	Pass the argument unevaluated; no value restriction.
Whole result slot, in every declaration	Return the equation body as a template.
Inside a structural type or field	Every value, so `(ListOf Atom)` holds `(1 "a" foo)`.
Index of a declared family	The index `Atom` itself; families are invariant.
`_`, `%Undefined%`	A whole slot with no restriction; inside a type they admit nothing.

Families are invariant because nothing declares how `(List $a)` varies with `$a`: `(List Atom)` holds `(Cons 1 (Cons "a" Nil))`, built at index `Atom`, but not a value declared `(List Number)`. Function types are compared as written; there is no subtyping relation.

Parameters are proper. A signature parameter ranges over proper types only. Otherwise `Atom`, which every value inhabits, would solve every shared parameter, and `(same 1 "s")` would pass. The rule holds inside compound types too: given `(: wide (Index Atom Atom))` and `(: f (-> (Index $a $a) $a $a Bool))`, the call `(f wide 1 "s")` must not succeed by setting $\$a = \mathsf{Atom}$, and it is refused in every argument order. The runtime attaches a delayed proper-type obligation to each signature parameter. Two consequences follow. A fact `(: x Atom)` is no evidence, so `x` is just a `Symbol`. And a value declared `(List Atom)` meets a requirement `(List Atom)` but not `(List $a)`.

3 One call, one body computation

For known inputs $\bar{v}$, a signature $d = (p_1, \ldots, p_n \to p_r)$ admits the call with result r when

$$\mathrm{Joint}_d(\Gamma_0, \Gamma_1, \bar{v}, r) \iff \exists \theta : \mathsf{Proper}.\ \left(\bigwedge_i \Gamma_0 \vdash v_i \Leftarrow \theta p_i \right) \wedge \Gamma_1 \vdash r \Leftarrow \theta p_r,$$

where Γ_0 is read at entry and Γ_1 at return. Several declarations for one function form a disjunction of such formulas. They are alternative justifications for the same computation, so the body runs once. Entry keeps every declaration the inputs admit, and the result may be justified by any of them, not only by the first one found. The equations alone determine the order and number of answers.

Each checked function compiles to an entry predicate around a private worker that holds its equations. For `(: inc (-> Number Number))` the generated entry tests are native Prolog, and an unbound argument gets a frozen test:

```
inc(X, R) :-
    ( var(X) -> type_freeze(X, number(X)) ; number(X) ),
    '$body$9'(X, R),
    ( var(R) -> type_freeze(R, number(R)) ; number(R) ).
```

A shared parameter needs a frame that relates the arguments and the result. For `same` the entry is `types_enter([[T,T,'_']], [X,Y], F)`, then the worker, then `types_exit(F, R)`, then the `Bool` test on `R`. Argument modes are chosen before arguments are evaluated, and the arguments captured by a partial application are not evaluated again. Direct, indirect, partial and memoized calls all reach the same entry. The result is checked after the body, so a check never precedes the body's effects or exceptions.

Open values. An unbound argument produces a residual requirement, never an invented value. Type variables private to the check may be unified; program variables are not. One residual formula keeps correlated arguments and alternative declarations together. A binding fails when the evidence then known contradicts the formula; otherwise the requirement waits for later bindings. The check does not decide whether an open requirement can still be met.

Entry evidence and declaration edits. The parts of an input known at entry keep the evidence they had at entry; a part that is still open records its evidence when it binds. Nothing else of the declaration view is saved. If `old` and `new` both have type `A`, a call `(keep old)` under `(: keep (-> $a $a))` may remove `(: old A)` and return `new`: the input's entry evidence still gives `A`, and the result is checked against the declarations at return. When a function returns one of its inputs unchanged, the input's evidence is reused only if SWI-Prolog's generation counters show that no declaration changed during the call [5].

4 Implementation

File	Lines	Responsibility
`types.pl`	383	Declaration index, evidence, joint frames, proper parameters, local structural checks.
`type_compilation.pl`	234	Entry and worker generation, argument modes, tail and suffix reuse, retirement after edits.
`type_membership.pl`	77	Indexed clauses for closed membership, with the general relation as fallback.
`type_obligations.pl`	69	Attributes on open values: evidence cells, watchers, copying and release.

Line counts include comments. The old call-site checks are removed from the translator, which shrinks by 46 lines. The first three files share the runtime module, so generated names and the definition lifecycle are unchanged; the attribute code is its own module.

A frame is `choices(Options, Stage, Identity, Stamp, Values)`, and an empty option list means the frame is discharged. Each open program variable holds one evidence cell and a list of watching frames without duplicates. An input whose parameter occurs elsewhere only in the result constrains nothing until the result exists; it only has to record later bindings. A call that is still running already records every binding of the variables it watches, so a nested call on the same open value does not watch them again, and recursion on one open value takes linear rather than quadratic time. At return, anything still unchecked arms its own watchers. All of this state is undone by ordinary backtracking.

Answers are collected with ordinary `findall`: constraints on returned variables are copied with them, and constraints on purely local variables are dropped. Dropping them is sound only if those local constraints are satisfiable. Adding an atom to a space stores plain variables, as SWI-Prolog's assert does.

5 What is proved

The Lean library `Mettapedia.Logic.ConstraintTypes` [6] states the abstract model and proves, among others:

1. **Checker exactness.** `DeclarationTyping.check_iff` and `OpenTyping.compile_correct`: the executable checkers decide their declaration judgments.
2. **Staged exactness.** `CallTyping.frame_exact` and `PatternContracts.frame_exact`: one assignment serves both the input and the result conditions.
3. **Compiled membership.** `DeterminedMembership.compiled_membership_correct`, under its premise that fields determine membership; a nominal fact about a whole value is not constructor evidence.
4. **Unshared parameters.** `UnsharedParameters.waive_joint_iff`: a parameter that is an input's whole type and occurs nowhere else may go unchecked when every value has some proper type, which the canonical ground model provides. This is coverage, not parametricity.
5. **Cost laws.** `CheckCost.incremental_exact` and `entry_check_covers_suffixes` justify per-part structural checks and suffix reuse; `CheckingPatterns.matches_any` covers `Atom` inside a type.

These are theorems about abstract finite-tree models. No proof yet connects them to the source normalizer, the Prolog emitter, SWI's attributed variables or the interpreter as a whole, so those are tested, not proved. Checks that traverse structure reject cyclic terms; whole slots without a restriction do not inspect them.

6 Cost and compatibility

Checking every suffix of an n-element list from scratch costs $n(n+1)/2$ element checks. For structural types over primitives and `Atom`, a suspension checks each newly bound cell once. A self-call on `cdr-atom` of a list already checked against `(ListOf T)` skips that slot; the other checks remain. Residual formulas for nominal or overloaded types can still rescan values.

The table compares the same typed programs on both trees through compiled MeTTa calls, including stable's call-site checks, as medians of fifteen interleaved runs against `stable-fixed 5d4b75e`. The two complete examples include loading and assertions. Inference counts are less sensitive to machine load than CPU time.

	Inferences		CPU seconds	
Workload	stable	draft	stable	draft
10,000 scalar calls	90,005	60,005	.0026	.0023
Counting, 200,000 steps	2,000,014	1,000,009	.088	.016
Constructor walk, 400 cells	1,851,823	83,012	.159	.0041
Open-variable loop	90,014	70,025	.0036	.0016
`nilbc`	20,100,081	19,486,151	.583	.533
`he_minimalmetta`, no declarations of its own	24,199,749	20,844,814	.755	.662

`nilbc` keeps its declaration `(: bc (-> $a Nat $b $b))`, whose shared `$b` relates an open query to the answer; nested calls on one open query are linear. The draft was faster in each of the 15 paired `nilbc` runs. Ten small untyped programs have identical inference counts on both trees, and three complete untyped examples stay within 0.5%.

Behaviour changes relative to `stable-fixed` are deliberate:

Case	stable	draft
`(: id (-> Number Number))` called through a variable on `"bad"`	`("bad")`	`()`
The same input bound to `"bad"` after the call	`("bad")`	`()`
One input with two nominal types	`(7 7)`	`(7)`
Body `(+ $x 1)` on 2, its `Atom` result declared after the equation	`3`	`(+ 2 1)`
Declarations `Number->Atom` and `String->Number`, body `(+ 1 2)`, input `"s"`	`()`	`(3)`
`(ListOf Atom)` input on `(1 "a" foo)`	`()`	`(7)`
`(List $a)` input on a value declared `(List Atom)`	`(7)`	`()`

In the mixed case an argument is held, and a result quoted, only when every declaration says `Atom` at that position. Two example files change: `typed_partial_arguments` no longer repeats answers and effects, and `types_dependent` uses explicit guards for classifications derived through `get-type`.

Validation: 113 runtime, 49 definition-order and 10 parser tests pass, normally and under `-O`, and 177 example files with 786 assertions pass. Compiled and general membership agree on 1,650 type/value pairs, for values given complete and for values bound later, before and after declaration edits. The code is on the `typing/minimal-runtime-stable-20261105` branch of godelclaw/PeTTa.

References

[1] A. Mycroft and R. A. O'Keefe. *A polymorphic type system for Prolog.* Artificial Intelligence, 23(3):295–307, 1984. doi:10.1016/0004-3702(84)90017-1.

[2] J. Jaffar and J.-L. Lassez. *Constraint Logic Programming.* POPL, 1987, 111–119. doi:10.1145/41625.41635.

[3] SWI-Prolog reference manual. Attributed variables.

[4] SWI-Prolog reference manual. `freeze/2`.

[5] SWI-Prolog reference manual. Impact of transactions.

[6] MeTTapedia. Lean modules `Mettapedia.Logic.ConstraintTypes` and `Mettapedia.Languages.MeTTa.PeTTa.RuntimeTyping`, commit `63cddbdaa9`. github.com/godelclaw/MeTTapedia.